

JavaScript Notes — Chapter 6 & 7

DOM (Document Object Model) — Part 1 & Part 2

Connecting HTML with CSS and JS

Tag	Used for	Placed in
<style>	Connect HTML with CSS	<head> (last)
<script>	Connect HTML with JS	<body> (last)

Both are usually placed last in their section so the rest of the HTML has already loaded before the styling/script tries to act on it.

The Window Object

The window object represents an open window in the browser. It is the browser's object (not a JavaScript object) and is created automatically by the browser — you never create it yourself.

- It is a global object with lots of properties and methods.
- Everything global in the browser (document, alert, console, setTimeout, etc.) actually lives inside window.

What is the DOM?

When a web page loads, the browser creates a Document Object Model (DOM) of the page — a tree-like structure made of objects, where every HTML tag becomes a "node" Javascript can read and change.

The DOM is what lets JavaScript interact with a static HTML page and make it dynamic.

The DOM Tree

The window object sits at the very top, and everything branches down from it:

```
window
├── document
│   ├── html
│   │   ├── head
│   │   │   ├── link
│   │   │   ├── meta
│   │   │   ├── meta
│   │   │   └── title
│   │   └── body
│   │       ├── div
│   │       │   ├── img
│   │       │   ├── h1
│   │       │   └── p
```

```
■ ■■ div
■ ■ script
```

Extra: elements at the same level (like `img`, `h1`, `p` inside the same `div`) are called **siblings**. An element directly inside another (like `div` inside `body`) is a **child**, and the containing element is its **parent**.

Inspecting window and document

```
console.log(window.document); // prints the full HTML code (as text/markup)

console.dir(window); // prints window as an object: properties, methods, etc.
console.dir(document); // same, but for document
console.dir(document.body); // shows body's children: head, p, title, text, etc.
```

`console.log(window.document)` gives you the computed HTML code. `console.dir(...)` gives you the underlying object — useful to browse its available properties and methods interactively in DevTools.

The DOM is used to get HTML information, and we can make changes to the page using that information.

Example — Changing Background Color

```
document.body.style.background = "green";
```

This is dynamic change — JavaScript updating the live page through DOM manipulation, without reloading or editing the HTML file itself.

DOM Manipulation — Selecting Elements

There are several ways to select an element so you can read or change it:

Method	Selects by	Returns
<code>getElementById("myId")</code>	id	A single element
<code>getElementsByClassName("myClass")</code>	class	HTMLCollection (array-like)
<code>getElementsByTagName("p")</code>	tag name	HTMLCollection (array-like)
<code>querySelector("p")</code>	any CSS selector	The first matching element
<code>querySelectorAll("p")</code>	any CSS selector	A NodeList of all matches

```
document.getElementById("myId");
document.getElementsByClassName("myClass");
document.getElementsByTagName("p");
```

`querySelector()` and `querySelectorAll()`

`querySelector()` accepts the same syntax as CSS selectors (`#id`, `.class`, `tag`) and returns only the 1st matching element. `querySelectorAll()` returns all matches as a `NodeList`.

```
let el = document.querySelector("p"); // first on the page

let david = document.querySelector("#david"); // first element with id="david"

document.querySelectorAll("p"); // ALL elements → NodeList
```

Note: a `Nodelist` looks like an array (you can loop it with `forEach/for-of`) but it isn't a true JavaScript array — some array methods like `map()/filter()` won't work on it directly.

Reading and Setting Content — Properties

Once an element is selected, we can get or set its value using these properties:

Property	Returns / Sets
<code>tagName</code>	Returns the tag name of the element
<code>innerText</code>	Returns/sets the visible text content of the element
<code>innerHTML</code>	Returns/sets the plain text + all HTML content/tags inside the element
<code>textContent</code>	Returns/sets all text content, including hidden text

```
let h2 = document.querySelector("h2");

h2.innerText = h2.innerText + " from Apna College";
// appends text to whatever h2 already had
```

Practice Problem — Update Every .div

Select all elements with class "div" and give each one a unique new value:

```
let divs = document.querySelectorAll(".div");
let idx = 1;

for (let div of divs) {
  div.innerText = `new unique value ${idx}`;
  idx++;
}
```

Template literal ``new unique value ${idx}`` builds a different string on every loop by inserting the current `idx`.

Navigating the DOM Tree

Besides selecting by `id/class/tag`, you can move directly between related nodes once you have one element in hand:

Property	Gives you
<code>parentElement</code>	The element's parent node

children	All direct child elements
firstChild / firstElementChild	The first child node / first child element
lastChild / lastElementChild	The last child node / last child element

```
console.dir(document.body.firstChild);
document.querySelector("div").children;
```

Extra — Node types: when walking the tree, you may run into different **node types**, not just visible tags: an **elementNode** is a real HTML tag (like a div), a **textNode** is the raw text inside an element, and a **commentNode** is an HTML comment (<!-- like this -->). firstChild/lastChild can land on any of these, while firstElementChild/lastElementChild skip straight to element nodes only — usually what you want.

```
console.dir(document) → document is an object with its own
properties and methods, inspectable the same way.
```

DOM — Part 2

Attributes — `getAttribute()` and `setAttribute()`

```
element.getAttribute("attr"); // reads an attribute's value
element.setAttribute("attr", "value"); // sets/updates an attribute's value

let img = document.querySelector("img");
img.getAttribute("src"); // read the current image path
img.setAttribute("src", "new-photo.jpg"); // change the image shown
```

Style — Changing CSS from JS

`node.style` lets you read or change that element's inline CSS directly from JavaScript.

```
let div = document.querySelector("div");

div.style.backgroundColor = "green";
div.style.fontSize = "26px";
```

Note: CSS properties with a hyphen (background-color) become camelCase in JS (backgroundColor) since JS property names can't contain hyphens.

Changing Text Content

```
div.innerText = "Hello";
```

Inserting New Elements

Adding a brand-new element to the page takes 2 steps:

- 1 Create the new element.
- 2 Add it to the DOM at the desired position.

```
let el = document.createElement("div"); // step 1: create (not yet visible on page)
```

Step 2 — Positioning Methods

Method	Adds the new element...
node.append(el)	At the end, inside node
node.prepend(el)	At the start, inside node
node.before(el)	Just before node, outside it (as a sibling)
node.after(el)	Just after node, outside it (as a sibling)

append() and prepend() place the new element **inside** node, at the end or start of its existing children. before() and after() place it **outside** node, as a sibling right next to it.

```
let newBox = document.createElement("div");
newBox.innerText = "I'm new!";

let container = document.querySelector(".container");
container.append(newBox); // adds inside, at the end
// container.prepend(newBox); // adds inside, at the start
// container.before(newBox); // adds outside, right before container
// container.after(newBox); // adds outside, right after container
```

Deleting an Element

```
node.remove();
```

Removes that element (and everything inside it) from the page entirely.

```
let oldBox = document.querySelector(".old-box");
oldBox.remove();
```

Quick Reference

Task	Code
Select by id	document.getElementById("id")
Select by class (all)	document.getElementsByClassName("cls")
Select first match (CSS selector)	document.querySelector("sel")
Select all matches (CSS selector)	document.querySelectorAll("sel")
Get/set visible text	el.innerText
Get/set text + inner HTML tags	el.innerHTML
Get/set an attribute	el.getAttribute() / el.setAttribute()
Change CSS from JS	el.style.property = "value"

Create a new element	<code>document.createElement("tag")</code>
Insert inside (end / start)	<code>node.append(el) / node.prepend(el)</code>
Insert outside (before / after)	<code>node.before(el) / node.after(el)</code>
Remove an element	<code>node.remove()</code>